



케이녹 웹해킹 3회차 과제

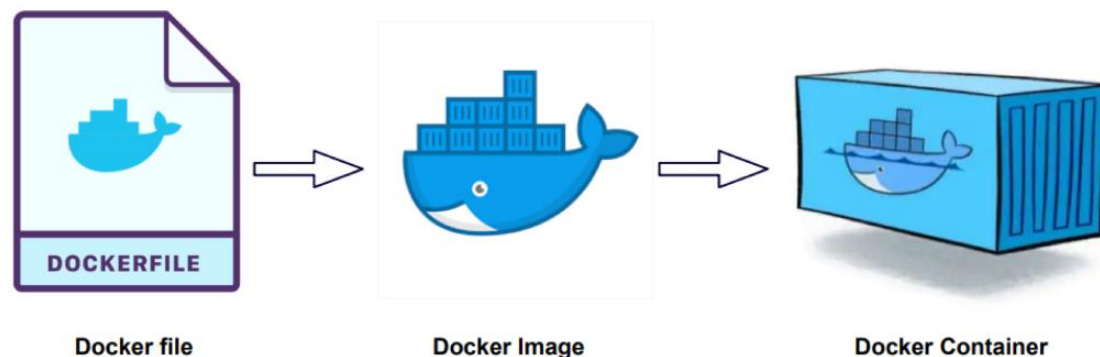
17기 최동현 (202511688)

1-1. Docker (Container)

• 도커(Docker)란?

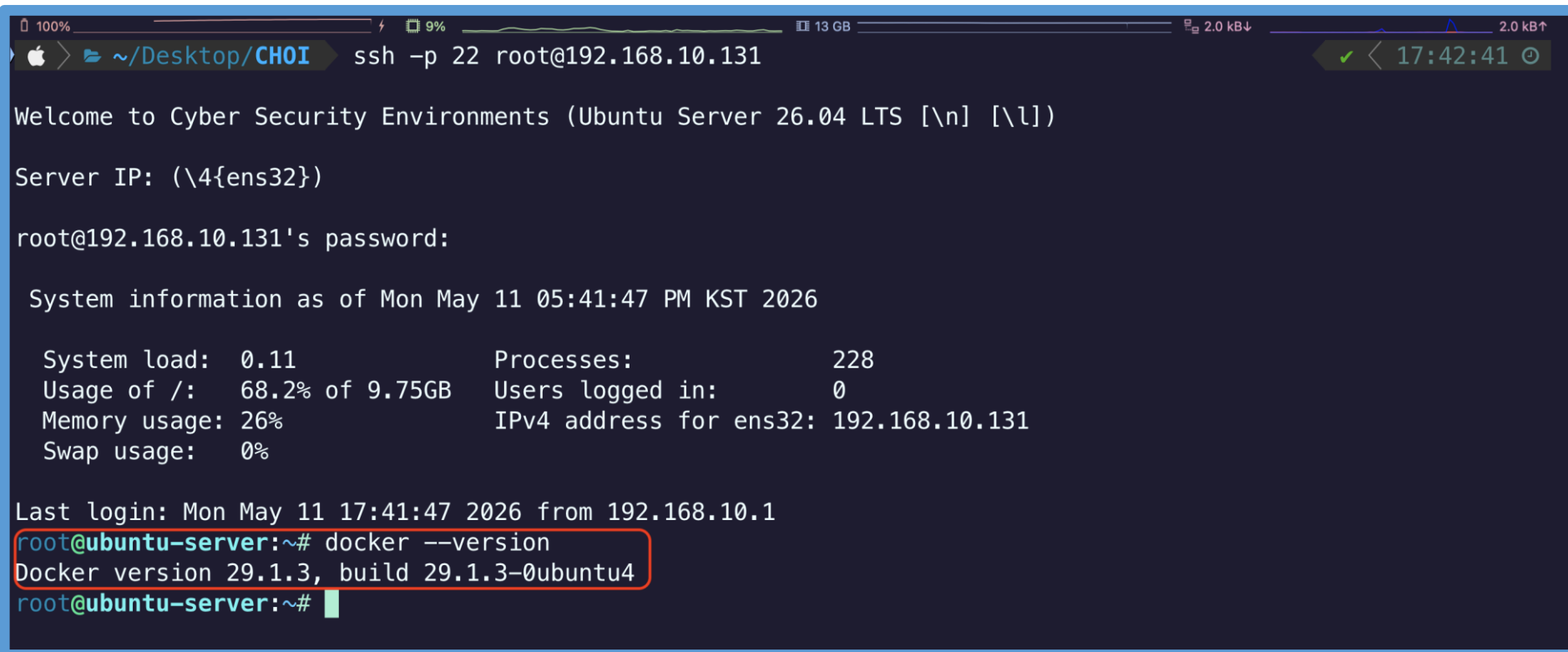
: Docker는 리눅스 커널의 Namespaces(Process, Network, Mount 등의 자원 격리)와 CGroups(CPU, 메모리 등의 물리적 자원 할당 제어) 기술을 활용하여, 호스트 운영체제 위에 독립된 애플리케이션 실행 환경인 컨테이너(Container)를 생성하는 OS 레벨 가상화 플랫폼입니다.

하이퍼바이저(Hypervisor)를 거치는 전통적인 가상 머신(VM)과 달리, 게스트 OS 없이 런타임 환경(Bin/Libs)만 격리하므로 Overhead(직접적인 기능이 아닌 자원)가 극히 적다.



1-2. Docker 설치

```
apt update -y
# Ubuntu 26.04 LTS (Debian 13) 기준
apt install docker.io docker-compose-v2 docker-buildx -y
```



The screenshot shows a terminal window with a dark background. At the top, there's a status bar with icons for battery (100%), network (9%), and storage (13 GB). The terminal prompt is `ssh -p 22 root@192.168.10.131`. The output shows a welcome message, server IP, password prompt, and system information. The system information includes system load, processes, memory usage, and IPv4 address. The last login is from 192.168.10.1. The user runs `docker --version` and the output is `Docker version 29.1.3, build 29.1.3-0ubuntu4`. The command and its output are highlighted with a red box.

```
100% 9% 13 GB 2.0 kB↓ 2.0 kB↑
~/Desktop/CHOI ssh -p 22 root@192.168.10.131 17:42:41
Welcome to Cyber Security Environments (Ubuntu Server 26.04 LTS [\n] [\l])
Server IP: (\4{ens32})
root@192.168.10.131's password:
System information as of Mon May 11 05:41:47 PM KST 2026

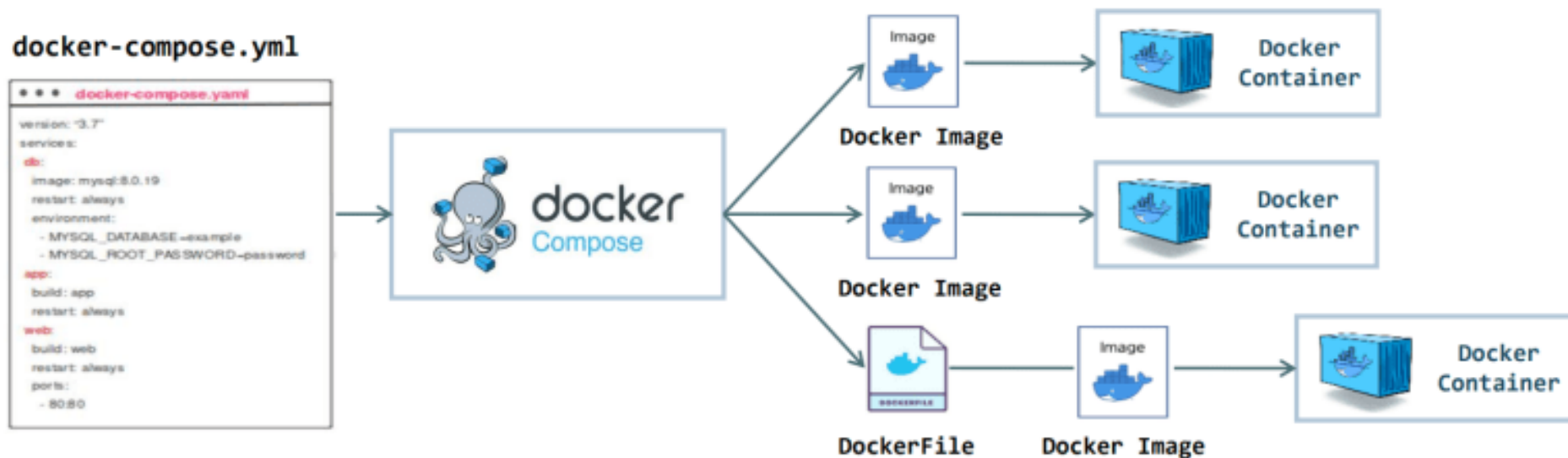
System load: 0.11      Processes:           228
Usage of /: 68.2% of 9.75GB Users logged in:    0
Memory usage: 26%      IPv4 address for ens32: 192.168.10.131
Swap usage: 0%

Last login: Mon May 11 17:41:47 2026 from 192.168.10.1
root@ubuntu-server:~# docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4
root@ubuntu-server:~#
```

1-3. Docker Compose

- Docker Compose란?

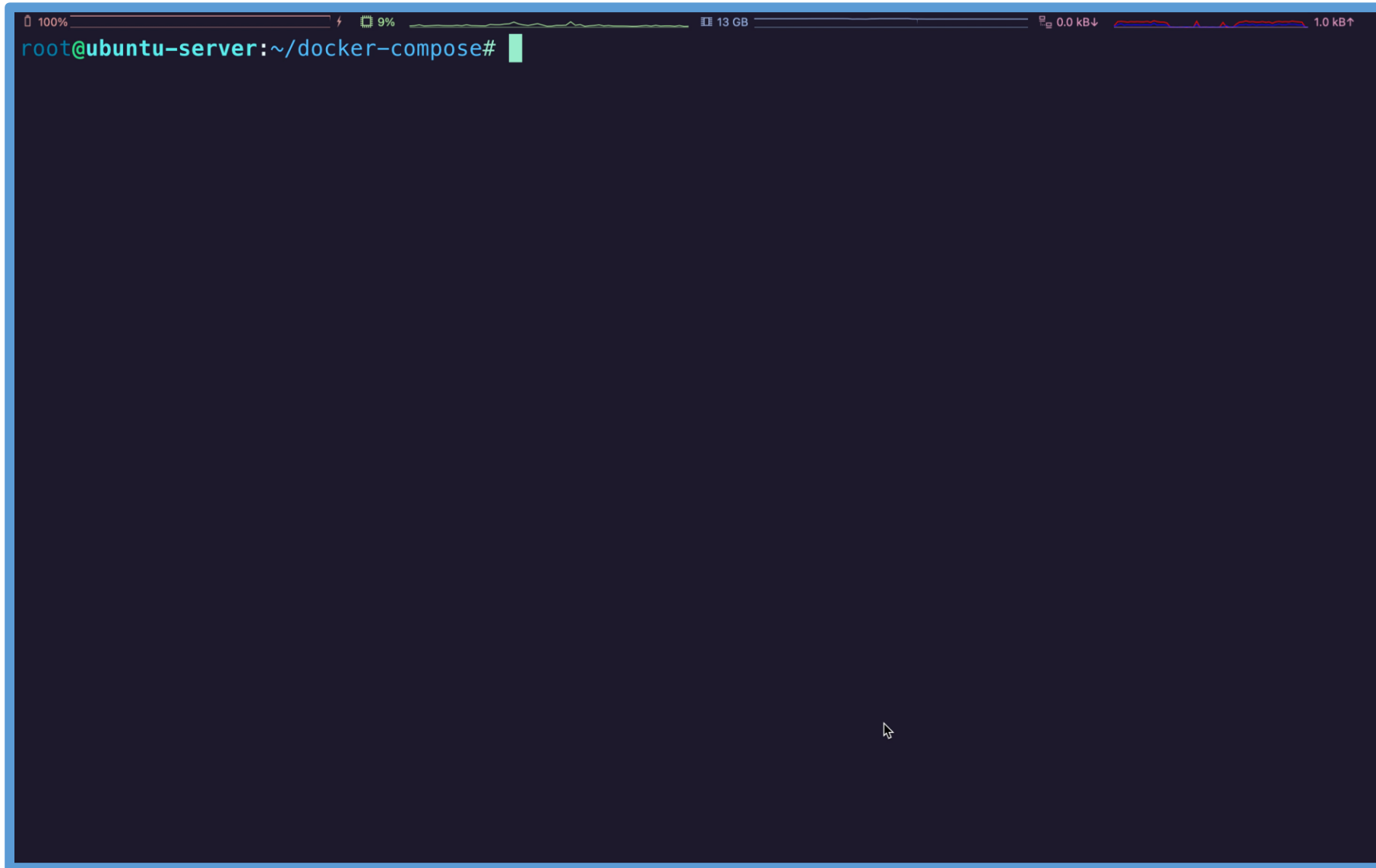
: **Docker Compose**는 단일 컨테이너 혹은 여러 개의 도커 컨테이너(Nginx 웹 서버, Spring Boot WAS, MySQL DB)로 구성된 다중 계층(Multi-tier) 아키텍처를 YAML (**docker-compose.yml**) 포맷으로 정의하고, 단일 명령어로 일괄 생성 및 네트워크를 연결(Orchestration)해 주는 도구입니다.



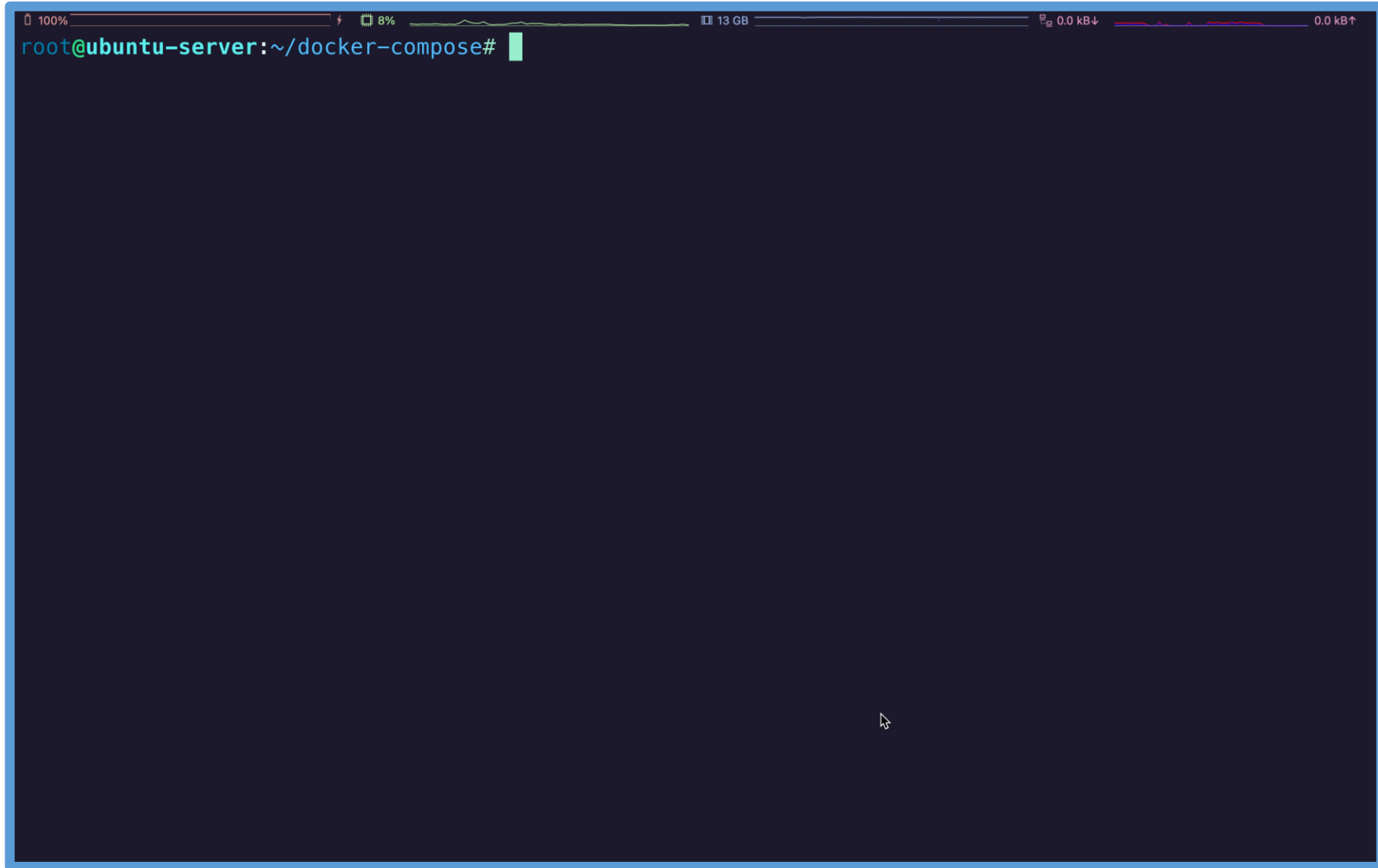
1-4. Docker 실습 (Dockerfile #1)

```
1 FROM debian:latest
2
3 ENV DEBIAN_FRONTEND=noninteractive
4
5 RUN apt update && apt install -y \
6     vim \
7     net-tools \
8     bind9-dnsutils \
9     tcpdump \
10    openssh-server \
11    && apt clean \
12    && rm -rf /var/lib/apt/lists/*
13
14 RUN echo "syntax on" >> /etc/vim/vimrc \
15     && echo "set number" >> /etc/vim/vimrc \
16     && echo "set ignorecase" >> /etc/vim/vimrc
17
18 RUN mkdir -p /run/sshd \
19     && sed -i 's/^#PermitRootLogin.*/PermitRootLogin yes/' /etc/ssh/sshd_config \
20     && sed -i 's/^PermitRootLogin.*/PermitRootLogin yes/' /etc/ssh/sshd_config \
21     && sed -i 's/^#PasswordAuthentication.*/PasswordAuthentication yes/' /etc/ssh/sshd_config \
22     && sed -i 's/^PasswordAuthentication.*/PasswordAuthentication yes/' /etc/ssh/sshd_config
23
24 RUN echo 'root:password' | chpasswd
25
26 COPY entrypoint.sh /entrypoint.sh
27 RUN chmod +x /entrypoint.sh
28
29 EXPOSE 22
30
31 CMD ["/entrypoint.sh"]
```

1-4. Docker 실습 (Dockerfile #2)



1-5. Docker 실습 (docker-compose #1)



2-1. Linux 명령어 (ls, cd, pwd)

```
100% 8% 13 GB
root@ubuntu-server:~/docker-compose# ls
Dockerfile  docker-compose.yaml  entrypoint.sh
root@ubuntu-server:~/docker-compose# ls -al
total 20
drwxr-xr-x 2 root root 4096 May 12 02:40 .
drwx----- 6 root root 4096 May 12 02:40 ..
-rw-r--r-- 1 root root 1392 May 12 02:21 Dockerfile
-rw-r--r-- 1 root root 293 May 12 02:36 docker-compose.yaml
-rw-r--r-- 1 root root 79 May 8 21:08 entrypoint.sh
root@ubuntu-server:~/docker-compose# cd ..
root@ubuntu-server:~# pwd
/root
root@ubuntu-server:~#
```


2-2. Linux 명령어 (cat, file, grep)

```
100% 10% 13 GB
root@ubuntu-server:~/docker-compose# cat file
K.knock
web
root@ubuntu-server:~/docker-compose# cat -n file
 1 K.knock
 2 web
root@ubuntu-server:~/docker-compose# file file
file: ASCII text
root@ubuntu-server:~/docker-compose# file kknock
kknock: directory
root@ubuntu-server:~/docker-compose# grep "web" file
web
root@ubuntu-server:~/docker-compose# cat file | grep "knock"
K.knock
root@ubuntu-server:~/docker-compose#
```

2-3. Linux 명령어 (find)

1. Find Command

HOST (Linux)

```
man find
```

- `-type`: 특정 파일만 찾아라.
 - `f`: 일반 파일 (File)
 - `d`: 디렉터리 (Directory)
 - `l`: 심볼릭 링크 (Symbolic Link)
 - `c`: 캐릭터 디바이스 파일 (Character special file: 키보드, 마우스 등 문자를 하나씩 입출력하는 장치)

```
find ./ -type f # 현재 디렉터리 하위로 파일만 전부 찾아라
```

- `-name`: 이름을 기준으로 찾아라.

```
find ./ -name "*.c" # 현재 디렉터리 하위로 확장자가 .c 인 파일만 찾아라  
find ./ -iname "*.c" # 위와 같은데, 대소문자 상관없이 전부 찾아라
```

- `-exec`: find로 찾아낸 각각의 대상(file, directory)으로 지정한 명령어를 즉시 실행해라.
 - `{}`: 찾아 낸 대상(파일, 디렉터리)를 담아두는 바구니

```
find ./ -name "*.c" -exec cp {} ./backup/{}.bak \; # 현재 하위로 찾은 C언어 파일들을 ./backup/ 디렉터리에 .bak 이라는 확장자로 복사한
```

2-4. Linux 명령어 (sort)

- -r: 정렬을 역순으로 해라
- -u: 중복된 값을 제거해라

```
~ / Desktop / CHOI cat ./Sort/file1.txt
```

```
line1  
line2  
line1  
line2  
line1  
line3
```

```
~ / Desktop / CHOI sort ./Sort/file1.txt
```

```
line1  
line1  
line1  
line2  
line2  
line3
```

```
~ / Desktop / CHOI sort -r ./Sort/file1.txt
```

```
line3  
line2  
line2  
line1  
line1  
line1
```

```
~ / Desktop / CHOI sort -u ./Sort/file1.txt
```

```
line1  
line2  
line3
```

```
~ / Desktop / CHOI
```

2-5. Linux 명령어 (uniq)

4. Uniq Command

HOST (Linux)

```
man uniq
```

- -c: 중복된 횟수 세기
- -d: 중복된 문자열만 보기 (Duplicate)
- -u: 중복되지 않은 문자열만 보기 (Unique)

```
~ / Desktop / CHOI cat ./Sort/file1.txt
```

```
line1  
line2  
line1  
line2  
line1  
line3
```

```
~ / Desktop / CHOI sort ./Sort/file1.txt | uniq -c
```

```
3 line1  
2 line2  
1 line3
```

```
~ / Desktop / CHOI sort ./Sort/file1.txt | uniq -d
```

```
line1  
line2
```

```
~ / Desktop / CHOI sort ./Sort/file1.txt | uniq -u
```

```
line3
```

```
~ / Desktop / CHOI
```

2-6. Linux 명령어 (strings)

```
man strings
```

- -n: 몇 글자 이상인 문자열만 출력
- -t x: 글자가 숨겨진 위치(문자열이 들어가는 주소) 찾기

```
file ./Pokemon
./Pokemon: Mach-O 64-bit executable x86_64

strings -n 20 -t x ./Pokemon
31fe K.Knock Pokemon Game
3215 press enter to start
322d [34m
3297 [33m
3301 [34m
336b [33m
33d5 [34m
343f [33m
34a9 [34m
3513 [33m
357d [34m
35e7 [33m
3651 [34m
36bb [33m
3725 [34m
378f [33m
37f9 [34m
3863 [33m
38cd [34m
3937 [33m
39a1 [34m
```

2-7. Linux 명령어 (base64, mkdir, rm)

```
root@ubuntu-server:~/docker-compose# echo "Welcome to K.knock!" | base64
V2VsY29tZSB0byBLLmtub2NrIQo=
root@ubuntu-server:~/docker-compose# echo "V2VsY29tZSB0byBLLmtub2NrIQo=" | base64 -d
Welcome to K.knock!
root@ubuntu-server:~/docker-compose# mkdir ./kknock
root@ubuntu-server:~/docker-compose# mkdir -p ./kknock/dir1/dir2/dir3
root@ubuntu-server:~/docker-compose# ls -lR ./kknock/
./kknock/:
total 4
drwxr-xr-x 3 root root 4096 May 12 03:16 dir1

./kknock/dir1:
total 4
drwxr-xr-x 3 root root 4096 May 12 03:16 dir2

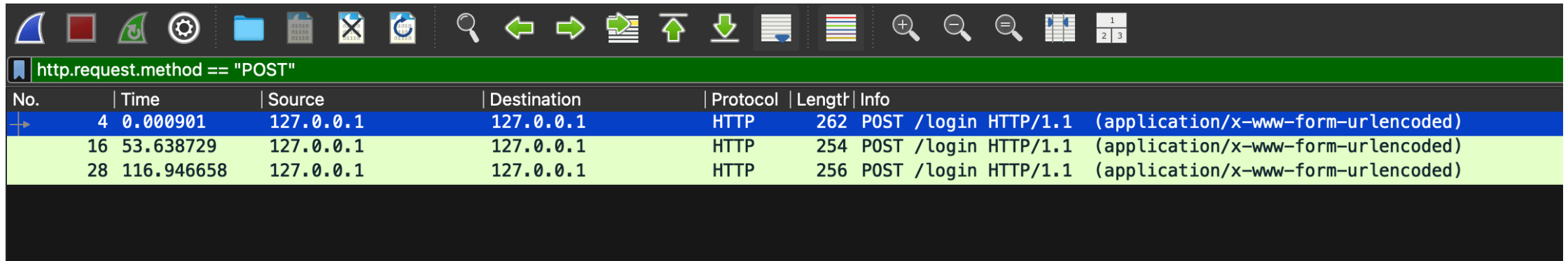
./kknock/dir1/dir2:
total 4
drwxr-xr-x 2 root root 4096 May 12 03:16 dir3

./kknock/dir1/dir2/dir3:
total 0
root@ubuntu-server:~/docker-compose# rm -rf ./kknock/
root@ubuntu-server:~/docker-compose#
```

2-8. Linux 명령어 (cp, mv, chmod)

```
100% 8% 13 GB 0.0 kB↓
root@ubuntu-server:~/docker-compose# echo "Welcome to K.knock!" | base64 > ./file
root@ubuntu-server:~/docker-compose# cp ./file ./copy_file
root@ubuntu-server:~/docker-compose# cat ./copy_file
V2VsY29tZSB0byBLLmtub2NrIQo=
root@ubuntu-server:~/docker-compose# cat ./copy_file | base64 -d
Welcome to K.knock!
root@ubuntu-server:~/docker-compose# mv ./copy_file ./rename_file
root@ubuntu-server:~/docker-compose# cat ./rename_file | base64 -d
Welcome to K.knock!
root@ubuntu-server:~/docker-compose# ls -l ./rename_file
-rw-r--r-- 1 root root 29 May 12 03:22 ./rename_file
root@ubuntu-server:~/docker-compose# chmod 600 ./rename_file
root@ubuntu-server:~/docker-compose# ls -l ./rename_file
-rw----- 1 root root 29 May 12 03:22 ./rename_file
root@ubuntu-server:~/docker-compose# su - cyber
cyber@ubuntu-server:~$ cat /root/docker-compose/rename_file
cat: /root/docker-compose/rename_file: Permission denied
cyber@ubuntu-server:~$
```

3. 실습 과제 풀이



No.	Time	Source	Destination	Protocol	Length	Info
4	0.000901	127.0.0.1	127.0.0.1	HTTP	262	POST /login HTTP/1.1 (application/x-www-form-urlencoded)
16	53.638729	127.0.0.1	127.0.0.1	HTTP	254	POST /login HTTP/1.1 (application/x-www-form-urlencoded)
28	116.946658	127.0.0.1	127.0.0.1	HTTP	256	POST /login HTTP/1.1 (application/x-www-form-urlencoded)

```
> Hypertext Transfer Protocol
< HTML Form URL Encoded: application/x-www-form-urlencoded
  < Form item: "username" = "kknock_admin"
    Key: username
    Value: kknock_admin
  < Form item: "password" = "K.knock2026!"
    Key: password
    Value: K.knock2026!
```

```
> Hypertext Transfer Protocol
< HTML Form URL Encoded: application/x-www-form-urlencoded
  < Form item: "username" = "imnotadmin"
    Key: username
    Value: imnotadmin
  < Form item: "password" = "hello123"
    Key: password
    Value: hello123
```

```
> Hypertext Transfer Protocol
< HTML Form URL Encoded: application/x-www-form-urlencoded
  < Form item: "username" = "testuser"
    Key: username
    Value: testuser
  < Form item: "password" = "test1234"
    Key: password
    Value: test1234
```

만약 HTTPS 통신이었다면? : POST Request 데이터가 TLS에 의해 암호화되어 전송되기 때문에 Sniffing tool로 중간에 확인 한다면 form값이 암호화되어 볼 수 없을 것이다.



감사합니다

17기 최동현